Interfaces et Modularité

Objectifs

1. Structurer un projet Go professionnel en plusieurs packages (`cmd/`, `internal/`) en exploitant les règles de visibilité par la casse (Majuscule vs minuscule).
2. Découpler des composants métiers grâce aux **interfaces implicites** définies du côté du consommateur (*"Accept interfaces, return structs"*).
3. Inspecter le type concret derrière une interface avec les *Type Assertions* (`val, ok := x.(T)`) et les *Type Switches*.
4. Reproduire, diagnostiquer et corriger le bug critique du pointeur typé `nil` stocké dans une interface (`iface`).

Exercice 1 : Architecture multi-packages et Interfaces implicites

Vous développez le système d'encaissement d'une plateforme e-commerce. Pour respecter les bonnes pratiques Go, l'orchestrateur de commandes ne doit jamais dépendre directement des implémentations concrètes des prestataires bancaires (Stripe, PayPal, ou un Mock de test).

1. **Mise en place de l'arborescence standard :** À l'intérieur de votre dossier `tp-j2s2`, créez l'arborescence de dossiers suivante :

```
None
tp-j2s2/
├── go.mod
├── cmd/
│   └── caisse/
│       └── main.go
└── internal/
    ├── commande/
    │   └── service.go
    └── passerelle/
        ├── stripe.go
        └── virement.go
```

2. **Package `passerelle` (Les producteurs concrets — Zéro interface déclarée ici !) :**

- Dans `internal/passerelle/stripe.go` (`package passerelle`), créez une structure exportée `ClientStripe` contenant un champ **privé** (non exporté) `cleSecrete string` et un champ **public** `ModeSandbox bool`.
- Ajoutez un constructeur exporté `NewClientStripe(cle string, sandbox bool) *ClientStripe`.
- Implémentez une méthode `Debiter(montant float64) (string, bool)` sur `*ClientStripe` qui affiche un message et retourne un identifiant de transaction (ex: `"STRIPE-TX-9981"`) et `true`.
- Dans `internal/passerelle/virement.go` (`package passerelle`), créez une structure `VirementSEPA` contenant un champ `IBAN string`, et implémentez également une méthode `Debiter(montant float64) (string, bool)` (sur *Value Receiver* ou *Pointer Receiver* de votre choix).

3. **Package `commande` (Le consommateur — Définit sa propre petite interface) :**

- Dans `internal/commande/service.go` (`package commande`), définissez une interface **à une seule méthode** nommée `Payeur` :

```Go
type Payeur interface {
    Debiter(montant float64) (idTransaction string, succes bool)
}
```

- Écrivez une fonction exportée `ValiderPanier(p Payeur, montantTotal float64)` qui appelle `p.Debiter(montantTotal)` et affiche le résultat.
- *Remarquez que le package `commande` n'importe jamais le package `passerelle`, et que `passerelle` n'importe jamais `commande` !*

4. **Point d'entrée `cmd/caisse/main.go` (Assemblage et vérification des *Method Sets*) :**

- Importez `"formation.go/j2s2/internal/commande"` et `"formation.go/j2s2/internal/passerelle"`.
- Tentez d'accéder directement à `stripe.cleSecrete` depuis `main.go` : vérifiez que le compilateur bloque l'accès à ce champ privé.
- Appelez `commande.ValiderPanier(...)` avec une instance de `*ClientStripe`, puis avec une instance de `VirementSEPA`.
- **Expérience sur les *Method Sets* :** Si `Debiter` est définie sur un *Pointer Receiver* `(c *ClientStripe)`, que dit le compilateur si vous tentez de passer une valeur déréférencée `*stripe` au lieu du pointeur `stripe` à `ValiderPanier` ? Pourquoi ?
- Exécutez le programme depuis la racine du projet avec `go run ./cmd/caisse`.

Exercice 2 : *Type Switch* et introspection d'interfaces

Le service d'audit souhaite enrichir la fonction `ValiderPanier` (ou créer une fonction dédiée `InspecterMoyenPaiement(p any)` dans `main.go`) pour appliquer des vérifications spécifiques selon le type concret ou les capacités supplémentaires du moyen de paiement reçu.

1. Ajoutez sur `VirementSEPA` une seconde méthode `DelaiTraitementJours() int` qui retourne `3`.
2. Dans `cmd/caisse/main.go`, créez une structure locale `MockPaiementTest struct{}` qui implémente `Debiter(montant float64) (string, bool)` pour simuler un test unitaire sans réseau.
3. Écrivez une fonction `DiagnostiquerPayeur(p commande.Payeur)` qui :
    - Utilise un **`switch v := p.(type)`** (*Type Switch*) pour identifier si `p` est un `*passerelle.ClientStripe` (auquel cas affichez la valeur de son champ `v.ModeSandbox`), un `passerelle.VirementSEPA` (affichez son `v.IBAN`), un `MockPaiementTest`, ou un type inconnu.
    - Utilise une **Type Assertion avec l'idiome "Comma Ok"** sur une interface anonyme (`if d, ok := p.(interface{ DelaiTraitementJours() int }); ok { ... }`) pour détecter dynamiquement si le moyen de paiement passé possède la capacité d'indiquer son délai de traitement, sans connaître son type concret !

Exercice 3 : Le piège mortel du pointeur `nil` dans une interface (`iface`)

Cet exercice met en évidence le bug le plus célèbre de Go en entreprise : retourner un pointeur typé `nil` à travers une interface.

1. Dans `cmd/caisse/main.go`, copiez et exécutez la fonction de fabrique suivante :

```go
func choisirPasserelleSecours(activerSecours bool) commande.Payeur {
    var clientSecours *passerelle.ClientStripe = nil // Pointeur typé initialisé à nil

    if activerSecours {
        clientSecours = passerelle.NewClientStripe("sk_live_secours", false)
    }

    return clientSecours // DANGER ICI !
```

```
}
```

2. Dans `main()`, appelez `secours := choisirPasserelleSecours(false)` (le mode secours est donc désactivé, aucun client n'a été créé).
3. Écrivez la vérification standard :

```Go
if secours != nil {
    fmt.Println("Passerelle de secours disponible, lancement du débit...")
    secours.Debiter(100.0)
} else {
    fmt.Println("Aucune passerelle de secours (nil).")
}
```

4. **Analyse et correction :**
    - Exécutez le programme : pourquoi entrez-vous dans le bloc `if secours != nil` alors que `clientSecours` vaut pourtant `nil` ?
    - Affichez le couple interne `(tab, data)` de l'interface avec `fmt.Printf("Type interne (tab) = %T | Valeur interne (data) = %v\n", secours, secours)`.
    - Corrigez la fonction `choisirPasserelleSecours` pour que `secours == nil` soit bien évalué à `true` lorsque `activerSecours` vaut `false`.